WDJ UNIT 3 QB ANS (2- marks)

Some questions have asked only for code still I have added answer for understanding

1. What are the phases of JSP lifecycle?

1. The first phase of the JSP lifecycle is **translation**, where the JSP file is converted into a Java servlet. This step happens when the JSP page is accessed for the first time.
2. The next phase is **compilation**, where the generated servlet code is compiled into a class file. This compiled class is then ready to be executed by the server.
3. The **initialization phase** occurs when the JSP container calls the jspInit() method. This method runs only once and is used to initialize resources required by the JSP page.
4. The **request processing phase** uses the jspService() method to handle client requests. It processes the request and generates the response for the client.
5. The final phase is **destruction**, where the jspDestroy() method is called. This method releases resources when the JSP page is removed from memory.

2. What is the role of servlet in JSP architecture?

1. In JSP architecture, the **servlet acts as the controller** in the web application. It manages the interaction between the user request and the application logic.
2. When a JSP page is requested, the **JSP engine converts the JSP into a servlet** before execution. This servlet then processes the request and generates the response.
3. The servlet container manages the **execution and lifecycle of servlets and JSP pages**. It ensures proper handling of requests and responses in the web application.
4. Servlets also help in **processing business logic and controlling application flow**. After processing, they forward the data to JSP pages for displaying results.

3. Explain scriptlet in JSP with code snippet

1. A **scriptlet tag in JSP is used to write Java code inside a JSP page**. It allows developers to execute Java statements directly in the JSP file.
2. Scriptlets are written between <% and %> tags. Any Java code written inside these tags is compiled and executed by the server.
3. The code written in scriptlet becomes part of the _jspService() method of the generated servlet. This method handles client requests and responses.
4. Scriptlets can include **variables, loops, conditions, and other Java statements**. They help in performing logic operations inside JSP pages.

Example:

```
<%
out.println("Welcome to JSP");
int a = 10;
%>
```

4. Explain declaration tag in JSP with code snippet

1. The **declaration tag in JSP is used to declare variables and methods** in a JSP page. These declarations become part of the servlet class generated from the JSP.
2. It is written using the syntax <%! ... %>. The declared variables and methods are placed outside the _jspService() method.
3. Declaration tags are useful when we need to define **class-level variables or helper methods** for use in the JSP page.
4. The declared variables can be accessed anywhere within the JSP page. This helps in sharing data across different parts of the page.

Example:

```
<%! int count = 0; %>
```

5. Explain expression tag in JSP with code snippet

1. The **expression tag in JSP is used to display the value of a variable or expression** directly in the output. It automatically sends the result to the response output stream.
2. The syntax of expression tag is <%= expression %>. It works like out.print() but does not require writing the print statement.
3. Expression tags are mainly used to **display values of variables, calculations, or method results** on a web page. They make JSP code shorter and easier to write.
4. The JSP container converts the expression into a print statement internally.

Example: <%= (2*5) %>

6. Differentiate between include directive and <jsp:include> action

1. The **include directive** includes another file during the translation phase of JSP. The included file becomes a permanent part of the JSP page before compilation.
2. The **<jsp:include> action** includes another resource during request processing time. It dynamically inserts the output of another file each time the page is accessed.
3. The syntax of include directive is <%@ include file="file.jsp" %>. The syntax of include action is <jsp:include page="file.jsp" />.
4. Include directive is mainly used for **static content**, while <jsp:include> action is used for **dynamic content** in web applications.

7. What is JavaBean? Explain Property of it.

1. A **JavaBean is a reusable Java class used to store and manage data in JSP applications**. It contains private variables and public getter and setter methods to access them.
2. JavaBeans are commonly used in JSP to separate business logic from presentation logic. They help in making web applications organized and reusable.
3. A **property in JavaBean refers to a variable whose value can be accessed using getter and setter methods**. These methods follow the naming convention getProperty() and setProperty().
4. JSP provides tags like <jsp:useBean>, <jsp:setProperty>, and <jsp:getProperty> to work with JavaBeans. These tags allow JSP pages to create objects and access their properties.

8. What is the role of JSP in MVC?

1. In the **MVC (Model-View-Controller) architecture, JSP mainly acts as the View component**. It is responsible for displaying the data to the user.
2. JSP receives processed data from the controller (usually a servlet). It then formats the data using HTML and JSP tags for presentation.
3. JSP helps in **separating presentation logic from business logic** in web applications. This separation makes applications easier to maintain and develop.
4. Using JSP as the view allows developers to design user interfaces easily. It works together with servlets and JavaBeans in MVC architecture.

9. What is the role of Maven in web applications?

1. **Maven is a build and dependency management tool used in Java web applications**. It helps in organizing project structure and managing libraries required for the project.
2. Maven uses a configuration file called **pom.xml** to define dependencies, plugins, and project details. This file automatically downloads required libraries from repositories.
3. Maven simplifies the **build process, compilation, testing, and packaging** of web applications. It reduces manual work required to manage project files.
4. It also ensures **consistent project structure and easier project management** for developers working in teams.

10. List any two JSP implicit objects

1. **Request Object** – The request object represents the client request sent to the JSP page. It is used to obtain user input, form data, cookies, and request parameters.

Example:

```
<%
String name = request.getParameter("username");
out.println("Hello " + name);
%>
```

2. **Session Object** – The session object is used to store user information across multiple pages of a web application. It helps maintain user data during a session.

Example:

```
<%
session.setAttribute("user","Meet");
String u = (String)session.getAttribute("user");
out.println(u);
%>
```

11. What is a directive tag?

1. A **directive tag in JSP is used to give instructions to the JSP container** about how the JSP page should be processed. It controls the overall behavior and settings of the JSP page.
2. Directive tags affect the entire JSP page during the translation phase. They help configure things like imports, session usage, and error handling.
3. Directive tags are written using the syntax <%@ directive %>. They are processed before the JSP page is compiled into a servlet.
4. Directive tags are important for **setting page configuration and including resources** in JSP applications.

12. Name the types of JSP directives.

1. **Page Directive** – This directive defines page-level instructions such as importing classes, session management, and content type. It controls the environment of the JSP page.
2. **Include Directive** – This directive includes the content of another file into the JSP page during translation time. It is commonly used to include header or footer files.
3. **Taglib Directive** – This directive is used to declare a custom tag library in JSP. It allows the JSP page to use custom tags defined in external libraries.
4. These directives help configure JSP pages and improve code reuse. They provide instructions to the server about how the JSP page should behave.

13. What is the use of page directive?

1. The **page directive is used to define attributes and settings for a JSP page**. It controls how the JSP container processes the page.
2. It can specify information such as imported classes, session usage, and content type. These attributes help configure the environment of the JSP page.
3. The syntax of page directive is <%@ page attribute="value" %>. Multiple attributes can be used within the same directive.
4. Common attributes include **import, session, contentType, errorPage, and language**. These attributes help manage page behavior and error handling.

14. Explain the difference between JSP and Servlet.

1. **JSP is mainly used for designing web pages with dynamic content**, while servlets are Java programs used to process requests and control application flow. JSP focuses on presentation.
2. JSP pages contain HTML with embedded Java code, whereas servlets are written completely in Java. Servlets generate HTML using print statements.
3. JSP is usually used as the **view component** in web applications. Servlets often act as the controller that handles business logic.
4. JSP code is first converted into a servlet before execution. Servlets are already compiled Java classes that run directly on the server.

15. Differentiate between forward and include actions.

1. The **<jsp:forward> action forwards the request from one page to another resource**. After forwarding, the original page stops processing.
2. The **<jsp:include> action includes the output of another resource** within the current JSP page. The original page continues execution after inclusion.
3. Forward action sends the request completely to another page, while include action only inserts the content of another resource. Both are used to manage page navigation in JSP.
4. Example syntax:

```
<jsp:forward page="home.jsp" />
```

```
<jsp:include page="header.jsp" />
```

16. Explain the purpose of include directive.

1. The **include directive is used to include the content of another file into a JSP page**. This inclusion happens during the translation phase before the JSP is compiled.
2. It is commonly used to reuse common components like headers, footers, or menus. This helps reduce code duplication in JSP pages.
3. The included file becomes a permanent part of the JSP page after translation. Any change in the included file requires recompilation of the JSP page.
4. This directive improves **code reusability and easier maintenance** of web applications.

17. Write syntax for page directive.

1. The **page directive is used to define attributes and configuration for a JSP page**. It tells the JSP container how the page should be processed.
2. The general syntax of the page directive is:

<%@ page attribute="value" %>

3. Multiple attributes can be used inside the same page directive. Common attributes include import, contentType, language, and session.
4. Example syntax with attributes:

<%@ page language="java" contentType="text/html" import="java.util.Date" %>

18. Write syntax to forward a request using JSP.

1. The **forward action in JSP is used to send a client request to another page or resource**. After forwarding, the control is transferred to the new page.
2. It is commonly used for navigation between JSP pages in web applications. The original page stops processing after forwarding the request.
3. The syntax to forward a request is:

<jsp:forward page="file.jsp" />

4. Example:

<jsp:forward page="home.jsp" />

19. How do you include a file using include directive?

1. A file can be included in a JSP page using the **include directive**. It inserts the content of another file during the translation phase.
2. This directive is useful for adding common components like header or footer files. It helps maintain consistent layout across multiple pages.
3. The syntax for including a file is:

<%@ include file="filename.jsp" %>

4. Example:

<%@ include file="header.jsp" %>

20. Analyze the advantages of JSP over Servlets.

1. JSP allows developers to **write HTML and Java code together easily**. This makes it simpler to design dynamic web pages compared to servlets.
2. In JSP, the presentation logic and business logic are separated more clearly. This separation improves readability and maintainability of code.
3. Servlets require many out.println() statements to generate HTML content. JSP avoids this problem by allowing direct use of HTML with embedded Java code.
4. JSP provides built-in features such as **implicit objects, directives, and action tags**. These features simplify web development and reduce coding effort.

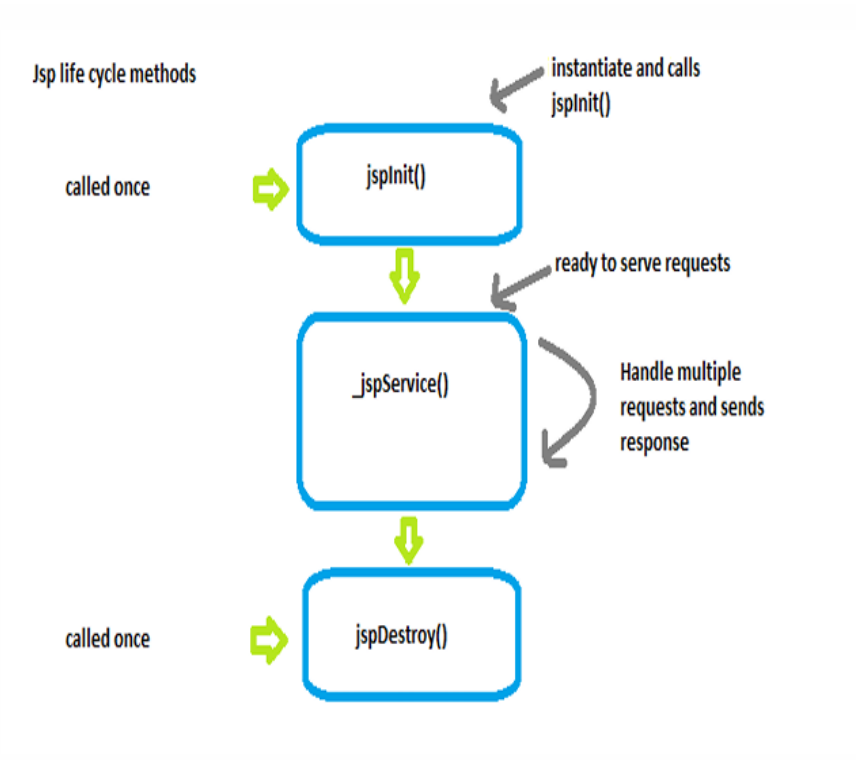
21. Discuss the advantages of using JavaBean

1. **JavaBeans provide reusable components** in JSP and web applications. Once created, the same bean class can be used in multiple pages and projects.
2. JavaBeans help in **separating business logic from presentation logic**. This separation makes JSP pages cleaner and easier to understand.
3. They use **getter and setter methods to access properties** of objects. This ensures proper data encapsulation and controlled access to variables.
4. JavaBeans are easy to integrate with JSP using tags like <jsp:useBean>, <jsp:setProperty>, and <jsp:getProperty>. These tags simplify object creation and property handling.

(5 – marks)

1. Explain the JSP lifecycle with diagram.

1. The **JSP lifecycle describes the stages a JSP page goes through from creation to destruction**. It explains how the JSP file is processed and executed on the server.
2. The first stage is **translation**, where the JSP page is converted into a servlet by the JSP engine. This happens when the page is requested for the first time.
3. The next stage is **compilation**, where the generated servlet code is compiled into a class file. This class file can then be executed by the server.
4. The **initialization phase** calls the jspInit() method only once when the JSP page is loaded. It is used to initialize resources needed by the page.
5. The **service phase** calls the _jspService(request,response) method for every client request. This method processes the request and generates the response.
6. The **destruction phase** calls the jspDestroy() method when the JSP page is removed from memory. It releases resources used by the JSP page.



2. Describe different JSP tags (scriptlet, expression, declaration) with examples.

1. **JSP scripting tags allow Java code to be written inside JSP pages**. They are mainly used to perform logic operations in the page.
2. **Scriptlet tag** is used to execute Java statements inside JSP. It is written between <% and %> tags.

```
<%
int a = 10;
out.println(a);
%>
```

3. Scriptlet code becomes part of the _jspService() method in the generated servlet. It is executed whenever the page is requested.
4. **Expression tag** is used to display the value of variables or expressions directly in the output. The syntax is <%= expression %>.

```
<%= (5*2) %>
```

5. Expression tags automatically print the result to the output stream. They do not require out.print() statements.
6. **Declaration tag** is used to declare variables or methods in JSP. It is written using <%! ... %>.

```
<%! int count = 0; %>
```

7. Declaration variables are placed outside _jspService() and can be used throughout the JSP page.

3. Explain various implicit objects in JSP.

1. **Implicit objects are automatically created by the JSP container** and can be used directly without declaration. They help developers access request, response, and session information.
2. The **out object** is used to send output to the client browser. It is an instance of JspWriter.

```
<%
out.println("Welcome to JSP");
%>
```

3. The **request object** represents the client request sent to the server. It is used to retrieve form data and request parameters.

```
<%
String name = request.getParameter("user");
%>
```

4. The **response object** is used to send responses back to the client. It can add cookies, redirect pages, or send status codes.
5. The **session object** stores user data across multiple pages. It helps maintain user information during a session.

```
<%
session.setAttribute("user","Meet");
%>
```

6. The **config object** is used to get initialization parameters for a JSP page. It is an instance of ServletConfig.
7. These implicit objects simplify JSP programming and provide easy access to web application resources.

4. Describe JSP directives in detail with proper examples.

1. **JSP directives are instructions given to the JSP container about how the JSP page should be processed.** They affect the whole JSP page and are processed during the translation phase.
2. There are **three main types of JSP directives: page, include, and taglib.** These directives help configure page settings and include external resources.
3. The **page directive** is used to define attributes such as language, content type, and imported classes. It controls the environment of the JSP page.

```
<%@ page language="java" contentType="text/html" import="java.util.Date" %>
```

4. The **include directive** is used to include another file in the JSP page during translation time. It is commonly used to include header or footer files.

```
<%@ include file="header.jsp" %>
```

5. The **taglib directive** is used to declare custom tag libraries in JSP. It allows developers to use predefined custom tags in JSP pages.
6. JSP directives help in **configuration, code reuse, and better organization** of web applications.
7. They are written using the syntax <%@ directive attribute="value" %>.

5. Write a JSP program to display current date and time.

1. JSP allows developers to **use Java code inside HTML pages to generate dynamic content**. It can display information such as the current date and time.
2. The program uses the **java.util.Date class** to obtain the current system date and time. This class must be imported using the page directive.
3. The JSP page prints the current date and time using the **out.println() method**. This method sends output to the browser.
4. The following JSP program displays the current date and time:

```
<%@ page import="java.util.Date" %>
<html>
<body>
<h2>Current Date and Time</h2>
<%
Date d = new Date();
out.println("Current Date and Time: " + d);
%>
</body>
</html>
```

5. When the JSP page is executed, the server processes the code and sends the result to the browser.
6. The user will see the **current system date and time displayed on the web page**.

6. Develop a JSP page to accept user input and display output.

1. A JSP page can **accept user input using HTML forms** and process the data using JSP code. This allows dynamic interaction between users and the web application.
2. The HTML form collects input from the user and sends it to another JSP page using the **action attribute**. The JSP page processes the request.
3. The **request implicit object** is used to retrieve the values entered by the user. The method request.getParameter() is used to get form data.
4. Example HTML form:

```
<form action="welcome.jsp">
Name: <input type="text" name="username">
<input type="submit" value="Submit">
</form>
```

5. JSP page to display output:

```
<html>
<body>
<%
String name = request.getParameter("username");
out.println("Hello " + name);
%>
</body>
</html>
```

6. When the user submits the form, the entered value is sent to the JSP page.
7. The JSP page processes the input and **displays the result dynamically in the browser**

7. Compare JSP vs Servlet with advantages and disadvantages.

1. **JSP is mainly used to create dynamic web pages**, while **Servlets are Java classes used to handle client requests and responses**. JSP focuses on presentation, whereas servlets focus on business logic.
2. JSP allows developers to **write HTML with embedded Java code**, making page design easier. In servlets, HTML content must be written using many out.println() statements.
3. An advantage of JSP is that it **separates presentation from business logic**, making the application easier to maintain. However, too much Java code inside JSP can reduce readability.
4. Servlets are **faster for processing complex logic** because they are pure Java classes. But writing and maintaining large HTML content in servlets is difficult.
5. JSP pages are **automatically converted into servlets by the JSP engine** before execution. Servlets are compiled Java classes that run directly on the server.
6. JSP is best for **user interface design**, while servlets are better for **handling application control and request processing**.

8. Describe all JSP action tags used for JavaBean integration with examples.

1. JSP provides **special action tags to work with JavaBeans**, which help manage data and business logic in web applications. These tags simplify object creation and property access.
2. The **<jsp:useBean> tag** is used to create or locate a JavaBean object in JSP. It links the JSP page with the JavaBean class.

```
<jsp:useBean id="user" class="com.model.User" scope="session" />
```

3. The **<jsp:setProperty> tag** is used to assign values to the properties of a JavaBean. It calls the setter method of the bean.

```
<jsp:setProperty name="user" property="name" value="Meet" />
```

4. The **<jsp:getProperty> tag** is used to retrieve and display values from a JavaBean property. It calls the getter method of the bean.

```
Name: <jsp:getProperty name="user" property="name" />
```

5. These tags allow JSP pages to **create objects, store data, and retrieve values easily**. They help keep business logic separate from presentation.
6. Using JavaBean integration makes JSP applications **more modular and reusable**.

9. Develop a JSP page that accepts form input and stores data in a JavaBean using <jsp:setProperty>.

1. In JSP, **JavaBeans are used to store and manage user data** in web applications. JSP action tags help connect the form input with the JavaBean object.
2. First, an **HTML form is used to accept user input** such as name or other details. The form sends the data to a JSP page for processing.

```
<form action="process.jsp">
Name: <input type="text" name="name">
<input type="submit" value="Submit">
</form>
```

3. In the JSP page, the **<jsp:useBean> tag creates the JavaBean object**. The bean class stores the data received from the user.

```
<jsp:useBean id="user" class="com.model.User" scope="request" />
```

4. The **<jsp:setProperty> tag stores the form data into the bean property**. It automatically calls the setter method of the bean.

```
<jsp:setProperty name="user" property="name" />
```

5. The stored value can be displayed using the **<jsp:getProperty> tag**.

```
Name: <jsp:getProperty name="user" property="name" />
```

6. This approach helps **separate data handling from presentation logic** in JSP applications.

10. Design a JSP-based application using JavaBeans for user registration form handling.

1. A **JSP-based user registration application uses HTML forms, JSP pages, and JavaBeans** to collect and store user information. The form collects data such as name, email, and password from the user.
2. The HTML form sends user data to a JSP page using the **action attribute and HTTP request method**. The JSP page processes the form data submitted by the user.

```
<form action="register.jsp">
Name: <input type="text" name="name">
Email: <input type="text" name="email">
<input type="submit" value="Register">
</form>
```

3. A **JavaBean class is created to store user details** using private variables and getter/setter methods. This bean acts as a data container for the registration information.
4. In the JSP page, the **<jsp:useBean> tag creates the JavaBean object** and connects it with the JSP page.

```
<jsp:useBean id="user" class="com.model.UserBean" scope="request" />
```

5. The **<jsp:setProperty> tag stores the form values in the JavaBean properties**. It automatically calls the setter methods of the bean.

```
<jsp:setProperty name="user" property="name" />
<jsp:setProperty name="user" property="email" />
```

6. The stored values can be displayed using **<jsp:getProperty> tag** or processed further for database storage. This structure keeps data handling separate from presentation logic.

11. Compare JSP and Servlet with advantages and disadvantages.

1. **JSP is used for designing dynamic web pages**, while **Servlets are Java programs used to process client requests and responses**. JSP focuses on presentation, whereas servlets focus on application logic.
2. JSP allows developers to **write HTML directly with embedded Java code**, making page development easier. In servlets, HTML must be generated using many out.println() statements.
3. An advantage of JSP is **better separation of presentation and business logic**, which improves readability and maintenance. However, excessive Java code in JSP can make pages complex.
4. Servlets are **more efficient for handling complex processing and control flow**. But writing large HTML pages inside servlets is difficult and less readable.
5. JSP pages are **automatically converted into servlets by the JSP engine before execution**. Servlets are compiled Java classes that run directly on the server.
6. JSP is mainly used for **user interface and display**, while servlets are used for **request processing and application control**.

12. Explain the need of JSP in web development.

1. JSP is needed in web development to **create dynamic web pages that can interact with users**. It allows web pages to display data generated from server-side programs.
2. JSP allows developers to **combine HTML with Java code**, making it easier to design dynamic web applications. This improves development speed and flexibility.
3. It helps **separate presentation logic from business logic** in web applications. This separation improves code organization and maintainability.
4. JSP provides features like **implicit objects, directives, action tags, and JavaBean integration**. These features simplify web application development.
5. JSP pages are **automatically compiled into servlets**, which makes them efficient and powerful for server-side processing.
6. Because of these features, JSP is widely used for **building scalable and interactive Java web applications**.

13. Explain the structure of a JSP page with example.

1. A **JSP page consists of HTML tags mixed with JSP elements** to create dynamic web content. It is saved with the .jsp extension and processed by the JSP container.
2. The structure usually contains **HTML code, JSP directives, scripting elements, and expressions**. These components allow the page to display dynamic data from the server.
3. JSP pages are translated into **servlets before execution** by the JSP engine. The servlet processes the request and sends the response to the browser.
4. Basic structure of a JSP page:

```
<%@ page language="java" contentType="text/html" %>
<html>
<head>
<title>JSP Example</title>
</head>
<body>
<%
out.println("Welcome to JSP");
%>
</body>
</html>
```

5. In this example, the **page directive defines page settings**, HTML creates the layout, and JSP scriptlet executes Java code.
6. This structure allows developers to easily combine **presentation and server-side processing** in one page.

14. Explain different JSP scripting elements with examples.

1. **JSP scripting elements allow Java code to be inserted into JSP pages**. They help perform logic operations and dynamic content generation.
2. The first type is the **Scriptlet tag**, which executes Java statements inside JSP. It is written between <% and %> tags.

```
<%
int a = 10;
out.println(a);
%>
```

3. The second type is the **Expression tag**, which displays the result of a Java expression directly in the output. It is written as <%= expression %>.

```
<%= (2*5) %>
```

4. Expression tags automatically print the result without using out.print(). They are commonly used to display variable values.
5. The third type is the **Declaration tag**, which is used to declare variables or methods in JSP. It is written using <%! ... %>.

```
<%! int count = 0; %>
```

6. These scripting elements make it possible to **combine Java programming with HTML in JSP pages**.

15. Explain JSP directives with examples.

1. **JSP directives provide instructions to the JSP container about how the page should be processed.** They affect the entire JSP page and are processed during translation.
2. There are **three main types of JSP directives: page, include, and taglib**. Each directive performs a specific configuration task.
3. The **page directive** defines attributes such as language, imported classes, and content type for the JSP page.

```
<%@ page language="java" contentType="text/html" import="java.util.Date" %>
```

4. The **include directive** is used to include another file into the JSP page during translation time. It helps reuse common components.

```
<%@ include file="header.jsp" %>
```

5. The **taglib directive** allows JSP pages to use custom tag libraries. It enables the use of predefined tags instead of Java code.
6. JSP directives help in **configuration, code reuse, and easier development of web applications**.

16. Describe how JSP integrates with Servlet in MVC pattern.

1. The **MVC (Model-View-Controller) pattern divides a web application into three parts: Model, View, and Controller**. This structure helps organize the application and makes development easier.
2. In this pattern, the **Servlet acts as the Controller** and manages client requests. It processes user input and controls the application flow.
3. The **Model contains the business logic and data**, often implemented using Java classes or JavaBeans. It stores and manages application data.
4. The **JSP acts as the View component** and is responsible for displaying data to the user. It formats the data received from the servlet using HTML and JSP tags.
5. When a user sends a request, the **Servlet processes the request and interacts with the Model**. It retrieves or updates data as required.
6. After processing, the servlet **forwards the request to a JSP page for displaying the result**. This separation improves maintainability and code organization.

17. Explain 5 JSP action tags with suitable examples.

1. **JSP action tags are XML-based tags used to control the behavior of the JSP engine.** They can include files, forward requests, or interact with JavaBeans dynamically.
2. The **<jsp:include> tag** includes another resource such as a JSP or HTML file at request time.

```
<jsp:include page="header.jsp" />
```

3. The **<jsp:forward> tag** forwards a request from one page to another resource.

```
<jsp:forward page="home.jsp" />
```

4. The **<jsp:useBean> tag** creates or locates a JavaBean object in JSP.

```
<jsp:useBean id="user" class="com.model.User" scope="session" />
```

5. The **<jsp:setProperty> tag** sets a value to a JavaBean property using its setter method.

```
<jsp:setProperty name="user" property="name" value="Meet" />
```

6. The **<jsp:getProperty> tag** retrieves and displays the value of a JavaBean property.

```
Name: <jsp:getProperty name="user" property="name" />
```

7. These action tags help perform **dynamic operations and simplify JSP programming**.

18. Explain pom.xml file and dependency management with example.

1. **pom.xml is the main configuration file used in Maven projects**. It contains information about the project, dependencies, plugins, and build settings.
2. Maven uses the **Project Object Model (POM)** to manage the project structure and automate the build process. All project details are defined inside the pom.xml file.
3. **Dependency management allows developers to add external libraries required for the project** without manually downloading them. Maven automatically downloads these libraries from repositories.
4. Each dependency is defined using group ID, artifact ID, and version. These identifiers help Maven locate the correct library.
5. Example of a dependency in pom.xml:

```
<project>
 <dependencies>
  <dependency>
   <groupId>javax.servlet</groupId>
   <artifactId>javax.servlet-api</artifactId>
   <version>4.0.1</version>
  </dependency>
 </dependencies>
</project>
```

6. Using pom.xml, developers can **manage dependencies, compile code, run tests, and package applications easily**.